

Parallel CSV Processing Framework.

Parallel CSV Processing Framework

Project Type

Data Processing / Performance Benchmarking

Main Technologies

- Python
 - Pandas
 - CSV
 - Multithreading
 - Multiprocessing
 - Benchmarking
-

1. Project Overview

The project is about processing **hundreds of large CSV files** through a data-processing pipeline.

The pipeline performs:

```
graph TD; A[CSV Files] --> B[Load]; B --> C[Validate]; C --> D[Clean]; D --> E[Transform]; E --> F[ ]
```

Aggregate
↓
Final Result

The main objective is not only to process the files, but also to compare different processing approaches.

We will compare:

1. Sequential processing
2. Multithreading
3. Multiprocessing

The comparison will focus on:

- Execution time
- Resource usage
- Scalability
- Suitability for the workload

2. Problem Statement

Process hundreds of large CSV files through a pipeline that loads, validates, cleans, transforms and aggregates data. Implement and benchmark sequential processing, multithreading and multiprocessing. Compare execution time, resource usage and scalability, and explain why one approach is more suitable for the workload.

3. Why is this problem important?

Processing a small CSV file is easy.

For example:

1 CSV
↓

```
Read
↓
Process
↓
Output
```

But imagine:

```
500 CSV files
+
Each file is large
+
Each file requires multiple operations
```

Sequential processing could take a significant amount of time.

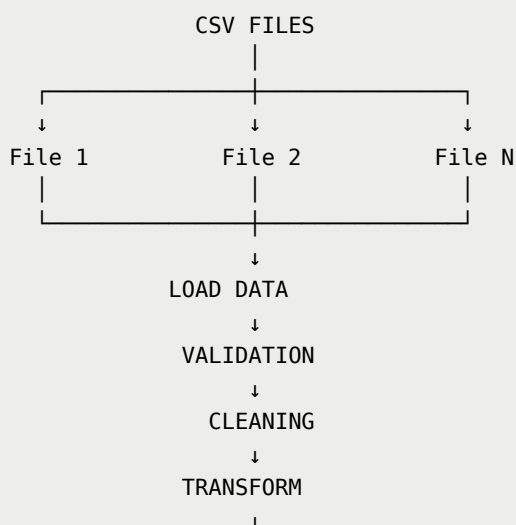
So the question becomes:

Can we process multiple files concurrently or in parallel to improve performance?

That is the main idea behind this project.

4. Project Architecture

I would represent the project like this:



AGGREGATION
↓
FINAL OUTPUT

5. Processing Pipeline

Step 1 — Load

Read the CSV file.

```
import pandas as pd  
df = pd.read_csv("data.csv")
```

Step 2 — Validate

Check whether the data satisfies expected conditions.

For example:

- Required columns exist
- Values are in expected formats
- Required fields are not missing
- Numeric values are valid

Example:

```
required_columns = ["name", "age", "salary"]  
if all(column in df.columns for column in required_columns):  
    print("Valid file")  
else:  
    print("Invalid file")
```

6. Cleaning

Cleaning removes or fixes unwanted data.

Examples:

- Remove duplicates
- Handle missing values
- Remove invalid rows
- Standardize text

Example:

```
df=df.drop_duplicates()df["name"]=df["name"].str.strip()
```

7. Transformation

Transformation changes data into the format required for further processing.

Example:

```
df["total"]=df["quantity"]*df["price"]
```

The exact transformations depend on the project's dataset.

8. Aggregation

Aggregation summarizes the processed data.

For example:

```
result=df.groupby("category")["total"].sum()
```

This could produce:

Category	Total
A	50000
B	72000
C	43000

9. Sequential Processing

The simplest approach is sequential processing.

Files are processed one after another.

```
File 1 → Process
      ↓
File 2 → Process
      ↓
File 3 → Process
      ↓
File 4 → Process
```

Example:

```
for file in files: process_file(file)
```

Advantage

Simple and easy to debug.

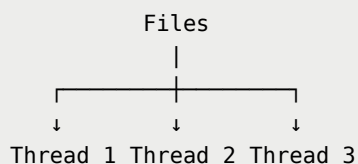
Limitation

When there are hundreds of files, the total execution time can become large.

10. Multithreading

With multithreading, multiple threads can work on different tasks.

Conceptually:



↓ ↓ ↓
File 1 File 2 File 3

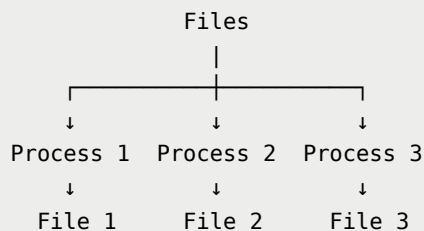
Python threads are particularly useful when the workload involves significant I/O waiting.

For example:

```
from concurrent.futures import ThreadPoolExecutor
with ThreadPoolExecutor(max_workers=4) as executor:
    results = executor.map(process_file, files)
```

11. Multiprocessing

Multiprocessing creates multiple processes.



Example:

```
from concurrent.futures import ProcessPoolExecutor
with ProcessPoolExecutor(max_workers=4) as executor:
    results = executor.map(process_file, files)
```

Multiprocessing can be useful when processing is CPU-intensive.

12. Why Python's GIL Matters

For standard CPython, the **Global Interpreter Lock (GIL)** limits execution of Python bytecode by multiple threads at the same time within a process.

This is one reason why:

```
CPU-heavy Python work
    ↓
Threads may not provide the expected CPU parallelism
    ↓
Multiple processes can be more suitable
```

However, this does **not** mean multithreading is useless.

For I/O-bound workloads, threads can still be very useful because the program can make progress while another thread is waiting for I/O.

13. Sequential vs Multithreading vs Multiprocessing

Feature	Sequential	Multithreading	Multiprocessing
Complexity	Low	Medium	Medium/High
Number of workers	1	Multiple threads	Multiple processes
I/O-heavy work	Limited	Often useful	Can be useful
CPU-heavy work	Limited	GIL can limit Python CPU parallelism	Often useful
Memory usage	Lower	Relatively lower	Higher
Debugging	Easier	More complex	More complex
Parallel CPU execution	No	Limited in CPython for Python bytecode	Yes

This table should be followed by **your actual benchmark results**, rather than claiming beforehand which method is fastest.

14. Benchmarking

The purpose of benchmarking is to measure how each approach performs.

We should measure:

Execution Time

How long does the complete pipeline take?

```
import time
start=time.perf_counter()# processing
end=time.perf_counter()
print(f"Time:{end-start:.2f} seconds")
```

CPU Usage

How much CPU does the approach use?

Memory Usage

How much RAM is consumed?

Scalability

What happens when the workload increases?

For example:

```
50 files
100 files
200 files
500 files
1000 files
```

15. Benchmark Table

Create this table in Notion and fill it with your actual results:

Method	Files	Execution Time	CPU Usage	Memory Usage
Sequential	100	—	—	—
Multithreading	100	—	—	—
Multiprocessing	100	—	—	—
Sequential	500	—	—	—
Multithreading	500	—	—	—

Method	Files	Execution Time	CPU Usage	Memory Usage
Multiprocessing	500	—	—	—

Don't put invented numbers here.

Your project becomes much stronger when you actually run the three approaches and record the results.

16. Scalability

Scalability means understanding how the system behaves as the workload increases.

For example:

```
100 files
  ↓
200 files
  ↓
500 files
  ↓
1000 files
```

We can observe whether execution time increases:

- Linearly
- Faster than expected
- More slowly than expected
- Or whether parallelism eventually stops helping

17. Resource Usage

Performance isn't only about execution time.

For example:

```
Method A
Time = 20 sec
```

```
Memory = 2 GB
```

```
Method B
```

```
Time = 22 sec
```

```
Memory = 500 MB
```

Method B may still be useful depending on the environment because it uses considerably less memory.

Therefore, the project compares:

| Speed + CPU + Memory + Scalability

rather than looking at only one number.

18. Possible Project Structure

I recommend using a structure like:

```
parallel_csv_framework/  
├── data/  
│   ├── input/  
│   └── output/  
├── src/  
│   ├── loader.py  
│   ├── validator.py  
│   ├── cleaner.py  
│   ├── transformer.py  
│   ├── aggregator.py  
│   ├── sequential.py  
│   ├── multithreading.py  
│   └── multiprocessing.py  
├── tests/  
│   ├── test_loader.py  
│   ├── test_validator.py  
│   └── test_transformer.py  
├── benchmarks/  
│   └── results.csv  
└── requirements.txt
```

```
|— README.md
|— main.py
```

This separates the processing logic from the benchmarking and tests.

19. Error Handling

Since hundreds of files are being processed, one bad file should not necessarily make the entire process fail.

For example:

```
def process_file(file_path):
    try:
        df = pd.read_csv(file_path)
        # validation
        # cleaning
        # transformation
        return df
    except Exception as e:
        print(f"Error processing {file_path}: {e}")
        return None
```

In a more complete implementation, use proper logging rather than relying only on `print()`.

20. Logging

For a larger data-processing framework, logging helps track what happened.

Example:

```
import logging
logging.basicConfig(level=logging.INFO, format="%(asctime)s - %(levelname)s - %(message)s")
logging.info("Starting CSV processing")
logging.info("Processing file: data1.csv")
logging.error("Failed to process data2.csv")
```

This becomes particularly useful when processing hundreds of files.

21. Unit Testing

The individual processing stages should be tested independently.

For example:

```
test_loader()
test_validator()
test_cleaner()
test_transformer()
test_aggregator()
```

Example:

```
def test_add(): assert add(2, 3) == 5
```

For validation:

```
def test_required_columns(): assert validate_columns(["name", "age"]) is True
```

22. Code Quality

The framework should follow the coding practices studied during the internship:

- PEP 8
- Meaningful variable names
- Functions with clear responsibilities
- Error handling
- Logging
- Unit tests
- Documentation
- Git version control
- Ruff
- Black

This is where your **Python research documentation and project documentation connect.**

23. Project Risks

Possible risks in this project:

Risk	Possible Impact	Mitigation
Very large CSV files	High memory usage	Chunk processing
Corrupted CSV	Pipeline failure	Exception handling
Different schemas	Validation errors	Schema validation
Too many processes	High memory usage	Control workers
Race conditions	Incorrect results	Safe shared-state design
Inconsistent data	Wrong aggregation	Cleaning + validation
Slow disk	Parallelism may not help	Benchmark I/O

24. Expected Learning

Through this project, I expect to understand:

- How large CSV files are processed
- How Pandas handles structured data
- How to design a processing pipeline
- Difference between sequential and parallel processing
- Practical use of threads and processes
- CPU vs I/O workloads
- Benchmarking
- Memory usage
- Scalability
- Error handling
- Logging
- Unit testing
- Code quality

25. Final Project Understanding

This is the section I'd personally put at the end because it sounds much more like an **internship learning document**:

The main idea behind this project is not simply to make CSV processing faster. The goal is to understand how different execution models behave for a real data-processing workload.

First, I need to build a correct sequential pipeline. Once the sequential version works, I can use the same processing logic with multithreading and multiprocessing. I can then benchmark all three approaches using the same input data.

The benchmark results will help determine which approach is suitable for the actual workload. If the workload is mainly waiting for I/O, multithreading may be useful. If the workload involves significant CPU-intensive Python processing, multiprocessing may provide better CPU parallelism. The actual conclusion should come from the measurements rather than assuming one method is always faster.

Final Notion Structure

```
INTERNSHIP DOCUMENTATION
|
├── 🐍 01 – Python & Software Development
|   |
|   ├── Python Basics
|   ├── Functions & OOP
|   ├── File Handling
|   ├── Exception Handling
|   ├── NumPy & Pandas
|   ├── Selenium
|   ├── Regex
|   ├── Multithreading
|   ├── Multiprocessing
|   ├── Concurrency vs Parallelism
|   |
|   └── SDLC
```

- └─ Agile & Scrum
- └─ Git
- └─ Documentation
- └─ Risk Management
- └─ PEP 8
- └─ Error Handling & Logging
- └─ Efficient Code
- └─ Programming Principles
- └─ Unit Testing
- └─ Ruff
- └─ Black

📊 02 – Parallel CSV Processing Framework

- └─ Project Overview
- └─ Problem Statement
- └─ Requirements
- └─ Architecture
- └─ Processing Pipeline
- └─ Sequential Processing
- └─ Multithreading
- └─ Multiprocessing
- └─ Benchmarking
- └─ Resource Usage
- └─ Scalability
- └─ Testing
- └─ Error Handling
- └─ Risks
- └─ Final Results

📝 03 – Internship Overall Learning